

Introducción a la POO

Conceptos básicos

La programación orientada a objeto se basa en la observación de que, en el mundo real, los objetos se construyen a partir de objetos más pequeños. Sin embargo, la capacidad de combinar objetos es sólo un aspecto general de dicha programación. También incluye conceptos y características que hacen que la creación y el uso de objetos sea más sencilla y flexible. La clase es la más importante de estas características.

Una **clase** es una plantilla que se utiliza para crear múltiples objetos con características similares. Cuando escribe un programa en un lenguaje orientado a objetos, no define objetos individuales, sino que define clases de objetos. Al escribir un programa en Java, se diseña y construye un conjunto de clases. Cuando el programa se está ejecutando, los objetos se crean desde estas clases y se usan conforme se van necesitando. Su tarea como programador de Java es crear el conjunto adecuado de clases para llevar a cabo lo que su programa requiere. Por fortuna, no tiene que empezar de cero, cada versión de lenguaje Java incluye un grupo de clases que implementan la mayoría de la funcionalidad básica que necesitará. A estas agrupaciones se les denomina bibliotecas.

En general, cada clase que escriba en Java está compuesta de dos componentes: atributos y comportamiento. Los **atributos** (variables de clase) son las cosas individuales que diferencian una clase de objetos de otros y determinan la apariencia, estado y otras cualidades de esa clase. En una clase, los atributos están definidos por variables, cada objeto puede tener distinto valor para sus variables (variables de instancia o de objeto). El **comportamiento** es la manera en que una clase de objetos puede hacer cualquier cosa para sí o para otros objetos. El comportamiento de una clase determina qué objetos de esa clase hacen cambiar sus atributos, y también qué hacen cuando otros objetos les piden hacer algo. El comportamiento de una clase de objetos se implementa a través de métodos, estos se usan para realizar tareas específicas del mismo modo que las funciones se usan en otros lenguajes de programación.

A continuación vamos a crear una clase Yeti que reflejará al abominable hombre de las nieves, al que daremos 3 atributos (color, sexo, hambriento). También implementaremos un comportamiento para alimentarlo y comprobar su estado.

Abstracción

Por medio de la abstracción conseguimos no detenernos en los detalles concretos de las cosas que no interesen en cada momento, sino generalizar y centrarse en los aspectos que permitan tener una visión global del problema.

Encapsulamiento

Esta característica permite ver un objeto como una caja negra en la que se ha introducido de alguna manera toda la información relacionada con dicho objeto. Esto nos permitirá manipular los objetos como unidades básicas, permaneciendo oculta su estructura interna.

La encapsulación y la abstracción están representadas por la *clase*. La abstracción porque en ella se definen los atributos de un determinado conjunto de objetos con características comunes, y es una encapsulación porque constituye una caja negra que encierra tanto los datos que almacena cada objeto como los métodos que permiten manipularlos.

Herencia

Es un mecanismo que hace posible que una clase herede todo el comportamiento y los atributos de otra clase.

A través de la herencia, una clase tiene inmediatamente toda la funcionalidad de una clase existente. Por esto, las nuevas clases se pueden crear indicando únicamente en qué se diferencian de la clase existente.

Polimorfismo

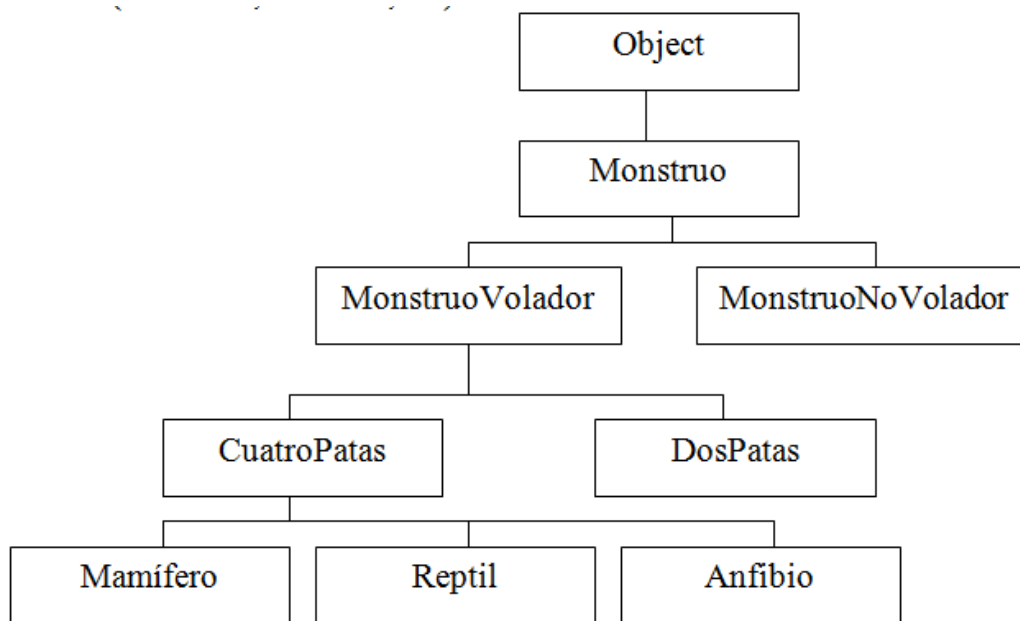
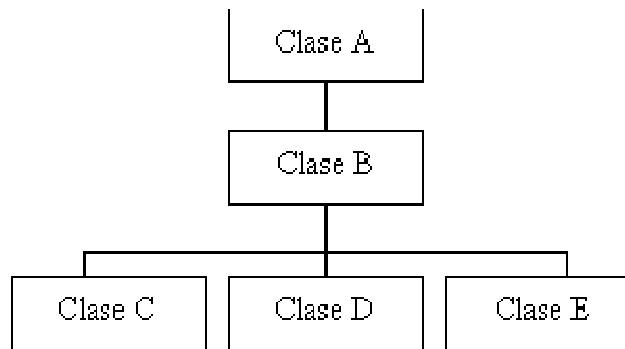
Permite implementar múltiples formas de un mismo método, dependiendo cada una de ellas de la clase sobre la que se realice la implementación. (Ej. Método sobrecargado)

Herencia y jerarquía.

La nueva clase se puede crear indicando en qué se diferencia de la clase existente.

A una clase que hereda de otra clase se le denomina **subclase**, y a la clase que proporciona la herencia se le llama **superclase**.

Una clase puede tener *sólo una* superclase pero puede tener un número ilimitado de subclases. Las subclases reciben por herencia todos los atributos y comportamiento de su superclase, es decir, la subclase recibe automáticamente todo esto de su superclase.



En la parte superior de la jerarquía de clases de Java está la clase **Object** (todas las clases heredan de esta superclase única). Conforme desciende la jerarquía, las clases se construyen para un propósito específico.

Una jerarquía de clases define conceptos abstractos en la parte superior de la jerarquía. Estos conceptos se vuelven más concretos conforme desciende la línea de subclases.

Jerarquía de clases.

Ventajas de organizar las clases en una jerarquía:

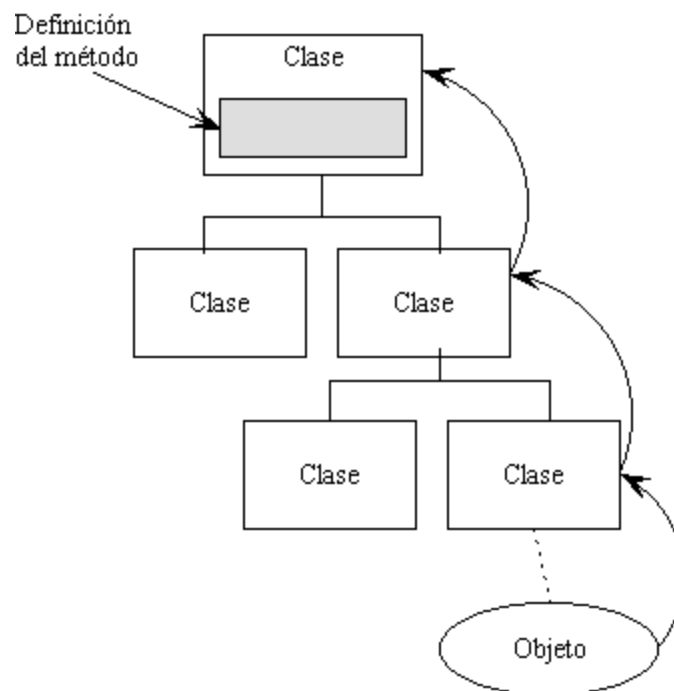
- La funcionalidad que es común a múltiples clases se puede colocar en superclases, lo que le permite ser utilizada repetidamente en todas las clases inferiores de la jerarquía.
- Los cambios a la superclase se reflejan automáticamente en todas sus subclases, las subclases de éstas y así sucesivamente. No hay necesidad

de cambiar o recompilar cualquiera de las clases inferiores; reciben la nueva información a través de la herencia.

Jerarquía y polimorfismo

Cuando se crea un objeto nuevo, Java registra cada variable definida para el objeto y cada variable definida para cada superclase del objeto. De este modo, todo lo que contienen las clases se combina para formar una estructura descriptiva del objeto en uso, y cada objeto aporta la información apropiada para su situación.

Los métodos trabajan de manera parecida:



Las cosas se complican cuando una subclase define un método que tiene el mismo nombre, tipo de valor de retorno y argumentos que otro método definido en una superclase.

¿Es lo mismo herencia y package?

Existen una serie de paquetes incluidos en el lenguaje (ver jerarquía de clases que aparece en el **API** de Java). Además el programador puede crear sus propios paquetes.

La jerarquía de las clases en Java.

Durante la generación de código en Java, es recomendable y casi necesario tener siempre a la vista la documentación on-line del **API de Java**. En dicha documentación es posible ver tanto la jerarquía de clases, es decir la relación de herencia entre clases, como la información de los distintos packages que componen las librerías base de Java.

- o **Package index**
- o **Class hierarchy**

Si se selecciona una clase en particular, la documentación muestra una descripción detallada de todos los métodos y variables de la clase. A su vez muestra su herencia completa (partiendo de la clase **java.lang.Object**)

Las clases en Java están contenidas en un paquete llamado **java**. El paquete **java** contiene paquetes más pequeños que define subconjuntos específicos de la funcionalidad del lenguaje Java, como:

- características estándar
 - manejo de archivos
 - multimedia
 - muchas otras cosas.
- De forma predeterminada, las clases que realiza el programador, sólo tienen acceso directo a las clases en **java.lang**. Para usar clases de cualquier otro paquete, tiene que referirse a ellas explícitamente por nombre de paquete o importarlas a su archivo fuente.
 - Para referirse a una clase dentro de un paquete, se deben listar todos los paquetes que la clase contiene seguidos del nombre de clase, con cada elemento separado por puntos(.).
Por ejemplo, para referirse a la clase *Color*, que está contenida en el paquete **awt**, el cual está a su vez en el paquete **java**: *java.awt.Color*